

Project Part 1 – Cloud-Deployed Data Application

1. Requirements

Develop a simple cloud-hosted data-driven web application consisting of a frontend and a backend, deployed using a Platform-as-a-Service (PaaS) solution. The system should be integrated with a version control repository and automated using a CI/CD workflow.

1.1 Functional Requirements

- Build a minimal backend that exposes at least one REST endpoint (e.g., /api/data).
- Build a minimal frontend that dynamically consumes the backend API and displays results (list, table, or visualization).
- Deploy both backend and frontend on one of the supported cloud providers Azure App Service/AWS Amplify/GCP App Engine.
- Integrate the repository with GitHub Actions so that every commit triggers build and deployment.
- Ensure both frontend and backend are reachable online (provide working public URLs).

1.2 Implementation Technologies

- Backend: Python (Flask, FastAPI) / Node.js (Express)
- Frontend: React / Streamlit / simple HTML+JS client
- Deployment: Azure App Service / AWS Amplify / GCP App Engine
- CI/CD: GitHub Actions

1.3 Non-Functional Requirements

- Version control using Git + GitHub.
- Automated deployment pipeline.

- Clear documentation of steps needed to build and run the application.

2. Deliverables

- Solution description document containing:
 - a) Conceptual architecture of the system (frontend + backend).
 - b) UML Deployment diagram.
 - c) README file with build and execution instructions.
- Source files hosted on GitHub repository with the exact format:
 - CC2025_Group_LastName_FirstName_Part_1

The repository must include both code and documentation (do not upload archives).

- Live links:
 - Provide live links to both frontend and backend if deployed separately.
 - If the frontend proxies requests to the backend, a single public link is sufficient but must demonstrate correct data fetching.

3. Evaluation

3.1 Project Related Basic Questions

During evaluation you may be asked about:

- Cloud service models (IaaS, PaaS, SaaS)
- REST API basics
- GitHub Actions (workflow triggers, jobs, runners)
- Deployment concepts (build vs release, environment variables)
- HTTP requests/responses
- CI/CD principles

3.2 Grading

Points	Requirements
5 p	Minimum to pass: deployed frontend + backend reachable online + GitHub repository with code + basic documentation.
1 p	Functional REST API in backend (CRUD or data retrieval).

1 p	Frontend consumes backend API dynamically and displays results.
2 p	Correct integration of GitHub Actions (pipeline runs automatically on push, deploys both frontend and backend).
1 p	README file with working URLs, architecture diagram, and usage instructions.
1 p	Code quality and repository organization (clean commits, proper structure).

Total: 10 points

Bonus (up to +1 p): If deployment of both frontend and backend is fully automated using Infrastructure-as-Code (Terraform) to provision and configure the cloud resources.

4. Bibliography

1. Microsoft Learn – [Azure App Service Documentation](#)
2. AWS Amplify – [Getting Started Guide](#)
3. Google Cloud – [App Engine Quickstart](#)
4. GitHub Docs – [GitHub Actions](#)
5. Streamlit Documentation – [Streamlit.io](#)